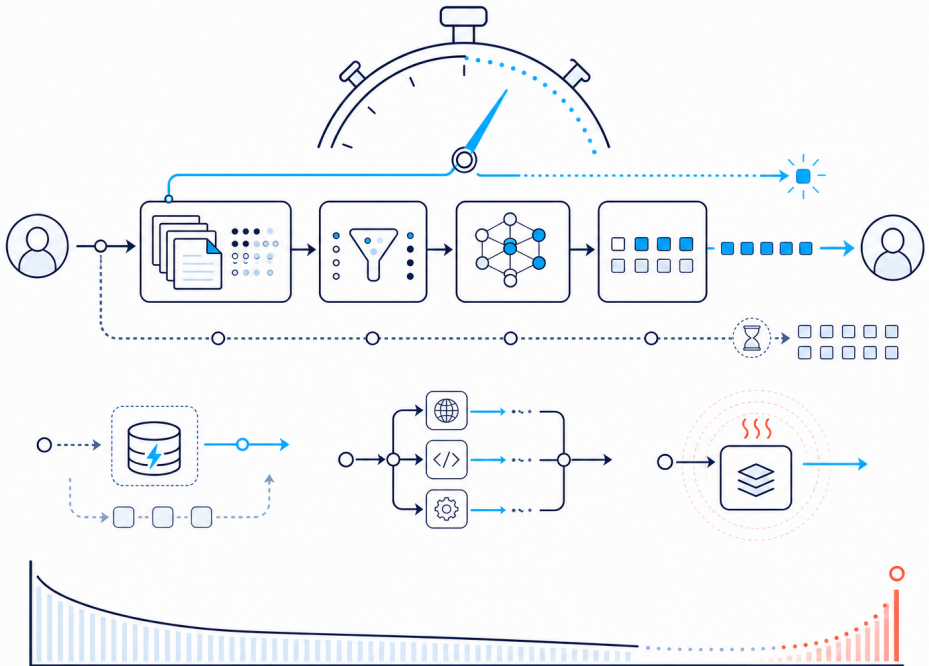


Latency For RAG & Agents



A 2-second answer that starts streaming at 300ms feels instant.
The same answer delivered all at once **feels broken**.



Latency Is A Product-Killer, And The Number Is P95

PROBLEM

Average latency lies. Half your users are slower than it, and the slow ones are the ones who leave.

THE AVERAGE HIDES THE TAIL



Past a couple of seconds with no response, users assume it failed and leave. **The tail is where users churn.**

Optimize the **tail (P95, P99)**, not the mean. The mean is a comfort.



AI Coach John

FOLLOW



REPOST

Optimize Perceived Latency, **Not** Total

CORRECTION

A slower total response can feel faster. The user judges the wait before the first token, not the wait until the last.

NO STREAMING – one blank wait, then everything

no stream

blank screen – is it stuck?

answer

first word at 2.0s

STREAMING – first word early, the rest flows

stream

the return window is 30 days ...

first word at 0.3s



Same 2s total, opposite feel. The user judges **time to first token, not time to last.**



AI Coach John

FOLLOW

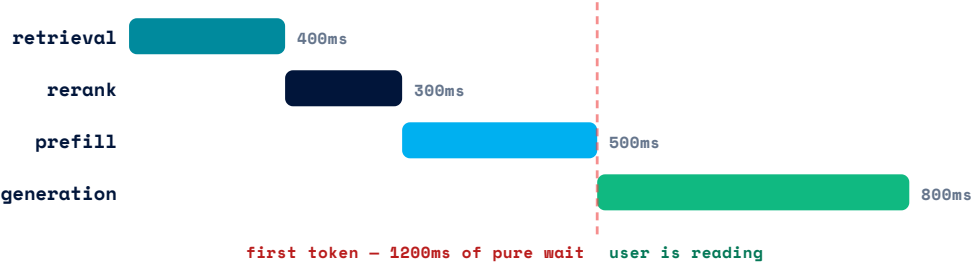


REPOST

Measure First, Trace The Turn

METHOD

The bottleneck is almost never where you think. For RAG it is usually retrieval and rerank, not generation.



STAGE	TIME	BEFORE FIRST TOKEN?
retrieval	400ms	yes — user waiting
rerank	300ms	yes — user waiting
prefill	500ms	yes — user waiting
generation	800ms	no — streams, user reads

🎯 1200ms of wait before the first word. That is the target, and it is not the generation.



Streaming

BIGGEST PERCEIVED WIN

Stream the answer token by token and the first word lands in a fraction of the total time. Highest impact, lowest effort.

⚡ SAME WORK, THE FIRST WORD ARRIVES EARLY

WITHOUT [===== compute 2s =====] then dump the whole answer

WITH [400ms] the ... return ... window ... is ... 30 ... days

First word early, and the user reads the rest as it flows.

low
effort

highest
perceived impact

- 1 Ship streaming **first**, before any other optimization. It changes the experience more than anything else on this list.



AI Coach John

FOLLOW



REPOST

Prompt Caching

CUT THE PREFILL

The stable front of your prompt is reprocessed on every request. Cache it and cut 200–400ms off time to first token.

►► CACHING CUTS THE PREFILL

UNCACHED [reprocess system + tools + context] → first token

CACHED [skip system + tools] [context] → first token

~200–400ms sooner, nearly free, and always correct.

↑↑ THE ONE RULE

Order stable-first: **system + tools cached at the front**, the retrieved chunks and query after. A changing byte near the front invalidates the cache after it.

- 2 Enable this **second**, right after streaming. Cheap, and it directly attacks the pre-first-token wait.



Fewer, Better Chunks

NO TRADEOFF

Retrieving 20 chunks instead of 5 makes the prompt longer, prefill slower, first token later. Better retrieval is faster AND cheaper.

20 CHUNKS

[long prompt] → slow prefill,
high cost, and a distracted model.

4 CHUNKS

[short prompt] → fast prefill,
low cost, a focused model.

↔ Retrieval quality and latency point the **same way**: fewer, more relevant chunks shorten the prompt, speed prefill, cut cost and often improve the answer. **The rare optimization with no tradeoff.**

🎯 The goal is not more context. It is **the right context**. Retrieve less, but retrieve better.



AI Coach John

FOLLOW

 REPOST

The Reranker Tax

RIGHT-SIZE IT

A reranker improves relevance and adds 200–400ms. Decide whether that trade is worth it, and where.

APPROACH	COST / QUALITY
rerank all 100 candidates	+400ms, best relevance
rerank top 20 only	+150ms, nearly as good
good embeddings, skip it	+0ms, fine for easy queries



Do not remove the reranker, **right-size it**: rerank fewer, use a smaller one, or skip it where the top result is already right.



AI Coach John

FOLLOW



REPOST

Cache Repeated Queries

THE REPEATS

A support agent answers the same fifty questions all day. Caching turns a repeat into an instant reply.

LAYER	SKIPS	RISK
retrieval cache	search + rerank	none (exact)
response cache	everything	threshold

i Use exact caching freely. Use a semantic response cache only where a wrong-but-similar hit is cheap.

% The multiplier is your **repeat rate**. FAQ agent → huge upside. Novel questions → little to cache.



Parallelize Tool Calls

BIGGEST AGENT WIN

An agent calling three tools one after another waits for all three in a row. If they do not depend on each other, call them at once.

SERIAL – each call waits for the last



PARALLEL – independent calls fire together



Independent calls run together: total = **the slowest, not the sum**. Only serialise calls that need a previous result.



AI Coach John

FOLLOW



REPOST

The Model Cascade

MATCH MODEL TO STEP

You do not need your best model for every step. Route the easy sub-tasks to a small fast model, and save the big one for hard generation.

STEP	MODEL	WHY
classify intent	small, fast	easy, do it in ms
rerank chunks	small, fast	a scoring task
final generation	large	needs the capability

Most steps in an agent are **easy**. Reserve the slow model for the hard one — the small model's speed compounds across many small calls.



Speed Up Generation Itself

INTER-TOKEN

Once the first token is out, inter-token latency decides how smooth it feels. Speculative decoding cuts the time between tokens.

→ STANDARD DECODING

one token per forward pass
steady, but slower

▶▶ SPECULATIVE DECODING

a draft model proposes N, the big model verifies in one pass
several tokens per step, no quality loss

→ Matters most for **long outputs**. Optimize TTFT first with streaming and caching, then reach for this.



AI Coach John

FOLLOW



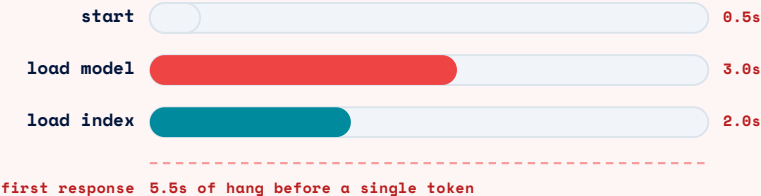
REPOST

Kill The Cold Start

FIRST REQUEST

If your service scales to zero, the first request pays seconds to load the model and index before it even starts working.

⌚ COLD START = SECONDS ON THE FIRST REQUEST



Keep **one instance warm**: no cold start, ever. Latency-sensitive and user-facing → worth the idle cost.



AI Coach John

FOLLOW



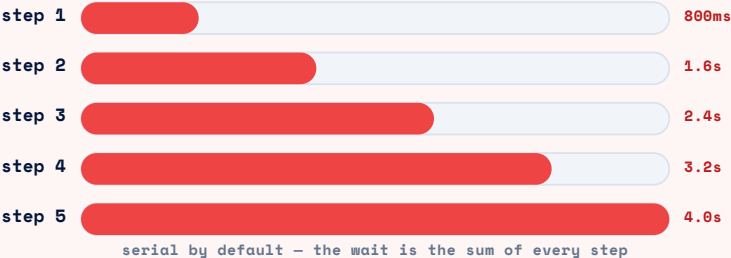
REPOST

Agents Multiply Latency

THE TRAP

Every step is a full round trip. A five-step agent at 800ms a step is a four-second wait, serial by default.

⚙️ LATENCY IS THE SUM OF THE STEPS



Cut latency by cutting **steps**: parallelise independent calls, cache repeated results, and design for fewer loops.



AI Coach John

FOLLOW



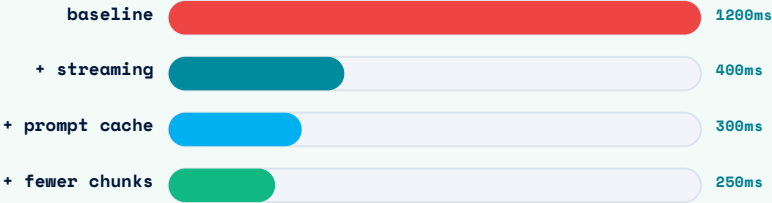
REPOST

The Levers Compound

THE PAYOFF

No single lever wins alone. Stack them and a 2-second turn becomes a sub-second first token.

↓ STACKING THE LEVERS ON A 2S TURN



time to first word – parallel tools collapse the agent steps behind it

✓ Compounded: **sub-second first token, smooth flow after.** Typical figures — trace your own pipeline.



AI Coach John

FOLLOW



REPOST

Set A Latency Budget Per Stage

THE TARGET

"Make it fast" is not a target. Assign a millisecond budget to each stage; any stage over budget is a bug.

STAGE	BUDGET	IF OVER
retrieval	< 150ms	cache, lower top-k
rerank	< 100ms	rerank fewer, or skip
prefill	< 200ms	prompt cache, fewer chunks
first token	< 450ms	the sum; the real target
inter-token	smooth	speculative decoding

 A budget turns "make it fast" into "this stage is 80ms over."

Which Lever First?

ORDER OF OPS

Apply them by impact over effort. The first three are cheap and win the most.

- 1 Streaming.** Low effort, highest perceived win. Always first.
- 2 Prompt caching.** Low effort, cuts TTFT. Enable next.
- 3 Measure + fix.** Trace the turn. Fix the biggest stage you find.
- 4 Fewer chunks** and, for agents, **parallel tool calls** — the biggest structural win.
- 5 Warm instance** if user-facing and cold starts are real.
- 6 Right-size rerank, model cascade, speculative decoding** once TTFT is handled.



Do **1 through 3** before anything else. They are cheap and win the most.



AI Coach John

FOLLOW



REPOST

What Is Your Bottleneck?

DECISION PATH

Trace one turn, then follow the biggest stage to its lever.

WHERE DOES THE TRACE SAY THE TIME GOES?

RETRIEVAL cache repeats, lower top-k, right-size the reranker

PREFILL prompt caching, fewer chunks, shorter system prompt

GENERATION speculative decoding, faster serving, shorter answers

TOOL CALLS parallelise independent calls, cache, fewer steps

FIRST REQUEST cold start — keep one instance warm

☰ And regardless of the bottleneck: **stream**, so the first token is early no matter what.



AI Coach John

FOLLOW



REPOST

Symptom To **Lever** DEBUG TABLE

What the user feels, and which lever fixes it.

SYMPTOM	CAUSE	LEVER
Long blank pause, then full answer	not streaming	streaming
Slow even on repeated questions	no caching	cache
First token slow, output fine	prefill / retrieval bound	top-k, cache
Fast average, some turns broken	P95 tail ignored	measure P95
Agent slow on multi-step tasks	serial tool calls	parallelise
First request of the day is slow	cold start	warm inst.
Long answers dribble out	inter-token latency	speculative
Big model for everything	no cascade	small model



First Token Fast, Then Never Stall

CHEATSHEET

The user experiences the wait before the first word and the flow after it. Optimize those two, in order.

LEVER	THE ONE LINE THAT MATTERS
streaming	first token early; the highest perceived win
prompt caching	cache the stable prefix; ~200-400ms off TTFT
fewer chunks	shorter prompt: faster, cheaper, focused
right-size rerank	pay the reranker tax only where relevance is hard
cache repeats	a repeated question should be instant
parallel tools	independent calls together; the sum becomes the slowest
model cascade	small fast model for the easy steps
speculative decode	faster inter-token flow for long outputs
warm instance	no cold start on the first user of the day
measure	trace the turn; optimise P95, not the average

⚡ First token fast, then never stall. That is the whole game.